Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»
КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА»

ОТЧЕТ
Предмет: Разработка интернет приложений
Лабораторные работы №6
“Асинхронная работа с данными”

Студент: Сигорский Александр Евгеньевич
Группа: ИБМ3-54Б
Преподаватель: М.В. Бурмистрова

Москва 2026

1. Описание задания

Цель лабораторной работы — рассмотреть асинхронную работу веб-приложения: использование Fetch API, Promise, async/await, параллельное выполнение запросов, обработку ошибок и обновление интерфейса после получения данных без полной перезагрузки страницы.

В CodeSlicer асинхронная модель используется во всей клиентской части. Построение карты проекта и review могут занимать заметное время, поэтому браузер не должен блокироваться до завершения серверной операции. JavaScript запускает HTTP-запрос, возвращает управление интерфейсу и продолжает работу после получения Promise.

CodeSlicer — local-first система анализа программного кода и влияния изменений. Интерфейс позволяет просматривать карту проекта, текущий review, локальные источники и дополнительные архитектурные представления. Визуально при оформлении проекта использовался подход современных AI-IDE, в том числе Kodik IDE.

2. Асинхронная модель CodeSlicer

Основной клиент для local API находится в frontend/api-client.js. Все методы ImpactApi либо сразу возвращают Promise от request, либо используют async-функции. В frontend/app.js данные загружаются только тогда, когда они необходимы конкретному экрану.

Асинхронность решает несколько практических задач:
• интерфейс остаётся отзывчивым во время HTTP-запроса;
• несколько независимых запросов можно выполнять параллельно;
• ошибки перехватываются через try/catch;
• результат можно сохранить в общем state и затем перерисовать только нужную часть DOM;
• пользователь может отменить длительный анализ.

3. Текст программы и пояснения

3.1. Асинхронная функция request

Листинг 1:

```
async function request(
    path,
    options
) {
    const response =
        await fetch(
            path,
            {
                cache: 'no-store',
                credentials:
                    'same-origin',
                ...options,
                headers: {
                    ...(options
                        ?.headers || {})
                }
            }
        );

    const text =
        await response.text();

    let data = {};

    try {
        data = text
            ? JSON.parse(text)
            : {};
    } catch (_) {
        data = {
            error: text
        };
    }
```

```
    if (!response.ok) {
        const detail =
            data.error
            || data.message
            || data.status;

        const error =
            new Error(
                detail
                || response.statusText
            );

        error.status =
            response.status;
        error.payload = data;

        throw error;
    }

    return data;
}
```

Ключевое слово async означает, что функция возвращает Promise. Оператор await применяется к fetch и response.text(). При этом браузерный поток событий не блокируется ожиданием сетевого ответа.

3.2. Асинхронный POST

Листинг 2:

```
const post =
    (path, payload) =>
        request(
            path,
            {
                method: 'POST',
```

```
        headers: {
          'Content-Type':
            'application/json',

          ...(sessionToken
            ? {
              'X-CodeSlicer-Session':
                sessionToken
            }
            : {})
        },

        body:
          JSON.stringify(
            payload || {}
          )
      }
    );
```

Функция post сама не объявлена async, потому что просто возвращает Promise от request. Это распространённый приём: если дополнительная обработка результата не требуется, Promise можно вернуть напрямую.

3.3. Параллельная загрузка через Promise.all

При начальном запуске необходимо проверить сервер и одновременно узнать текущее состояние проекта.

Листинг 3:

```
async function bootstrap() {
  try {
    const [
      health,
      current
    ] =
      await Promise.all([
```

```
        ImpactApi.health(),
        ImpactApi.state()
    ]);

if (health.status
    !== 'ok') {
    throw new Error(
        'local API не готов'
    );
}

const configuredProject =
    current.project_path
    || '';

const projectAvailable =
    current.project_exists
    !== false;

state.project =
    projectAvailable
        ? configuredProject
        : '';

state.hasAnalysis =
    Boolean(
        current.has_analysis
        && projectAvailable
    );

state.analysis =
    current.analysis
    || null;

state.ready = true;

if (state.hasAnalysis) {
```

```
            await hydrateAnalysis();
            renderRoute();
            showState(
               'Карта проекта готова.'
            );
         } else {
            renderRoute();
         }
      } catch (error) {
         state.ready = true;
         renderRoute();

         showState(
            `Локальный API недоступен:
            ${error.message}`,
            'error-state'
         );
      }
}
```

Promise.all запускает health и state одновременно. Если выполнять их последовательно, общее время ожидания было бы суммой времени двух запросов. Здесь приложение ждёт только завершения более долгого из двух запросов.

3.4. Promise.allSettled для частично доступных данных

В некоторых экранах CodeSlicer можно показать результат даже тогда, когда один дополнительный источник временно недоступен. Поэтому используется Promise.allSettled.

Листинг 4:

```
const results =
   await Promise.allSettled([
      ImpactApi.overview(),
      ImpactApi.state(),
```

```
        ImpactApi.progress(),
        ImpactApi.inventory(),
        ImpactApi.adapters()
    ]);

const [
    overview,
    current,
    progress,
    inventory,
    adapters
] = results;

if (overview.status
    === 'rejected') {
    throw overview.reason;
}

state.overview =
    overview.value || {};

state.inventory =
    inventory.status
        === 'fulfilled'
        ? inventory.value
            .inventory || {}
        : null;

state.adapters =
    adapters.status
        === 'fulfilled'
        ? adapters.value
            .adapters || []
        : state.adapters;
```

Отличие allSettled от Promise.all принципиально. Promise.all завершается ошибкой при первом rejected Promise. Promise.allSettled возвращает

состояние каждого запроса. Это подходит для обзорного экрана, где отсутствие одного дополнительного блока не должно уничтожить все остальные успешно полученные данные.

3.5. Асинхронная загрузка карты

Листинг 5:

```
async function loadProjection(
    force = false
) {
    if (!state.project) {
        return;
    }

    try {
        const filters = {
            node_kinds: [],
            edge_kinds: [],
            evidence_sources: [],
            min_confidence: 0
        };

        const response =
            await ImpactApi
                .graphProjection({
                    project_path:
                        state.project,

                    level:
                        state.mapLevel,

                    query:
                        $('#graphProjectionQuery')
                            ?.value.trim()
                        || '',
```

```
                filters,

                show_all:
                    state.mapScope
                        === 'all',

                max_nodes: 120,
                max_edges: 200
            });

        state.projection =
            response;

        renderProjection(
            response
        );

        updateEntitySuggestions(
            response.nodes || []
        );
    } catch (error) {
        showState(
            error.message,
            'error-state'
        );
    }
}
```

Пока выполняется запрос, пользовательский интерфейс остаётся активным. После await данные сохраняются в state.projection, и только затем вызывается renderProjection. Таким образом, визуализация появляется в момент, когда для неё действительно есть серверные данные.

3.6. Асинхронная загрузка review

Проверка изменений — ещё один пример операции, которая зависит от backend. Клиент получает структурированный отчёт, а затем строит карточки, таблицы и рекомендации.

Типовая схема:

```
async function loadReview(
   force = false
) {
   if (!state.project) {
      return;
   }

   try {
      const response =
         await ImpactApi.review({
            project_path:
               state.project,

            refresh:
               force
                  ? 'auto'
                  : 'never',

            max_results: 10,
            run_tests:
               'suggested'
         });

      state.review =
         dataPayload(response);

      renderReview();
   } catch (error) {
      showState(
         `Review не выполнен:
         ${error.message}`,
```

```
            'error-state'
        );
    }
}
```

Здесь async/await делает код похожим на последовательный, хотя фактически между отправкой запроса и получением ответа выполнение функции приостанавливается, а event loop продолжает обслуживать интерфейс.

3.7. Асинхронное подтверждение операции

Некоторые действия могут запускать локальный процесс. Для них CodeSlicer использует дополнительное подтверждение.

Листинг 6:

```
async function executeApprovedAction(
    callback,
    payload
) {
    const exactPayload =
        Object.freeze({
            ...(payload || {})
        });

    try {
        return await callback(
            exactPayload
        );
    } catch (error) {
        const pending =
            error
            && error.status === 409
            && error.payload
            && error.payload.status
                === 'pending_approval'
```

```
            ? error.payload
            : null;

        if (!pending) {
            throw error;
        }

        const token =
            await requestApprovalToken(
                pending
            );

        if (!token) {
            throw error;
        }

        const approvalId =
            pending.approval
                ?.approval_id;

        return callback({
            ...exactPayload,
            approval_id:
                approvalId,
            approval_token:
                token
        });
    }
}
```

В этой функции присутствуют два этапа ожидания: сначала выполняется действие, затем при необходимости ожидается ввод токена пользователем. Promise позволяет представить весь сценарий как одну асинхронную операцию.

3.8. Отмена длительного анализа

Backend хранит CancellationToken, a frontend имеет отдельную кнопку отмены. API-клиент предоставляет метод:

```
cancelAnalyze:
  () =>
    post(
      '/api/analyze/cancel',
      {}
    )
```

На backend:

```
def cancel_analysis(self):
  with self.lock:
    if not self.analysis_running          or self.cancellation is None:

      return {
        "status": "idle",
        "message":
          "No analysis is running",
        "progress":
          self.progress
      }

    self.cancellation.cancel()

    return {
      "status": "cancelling",
      "message":
        "Cancellation requested",
      "progress":
        self.progress
    }
```

Это важное дополнение к асинхронной работе. Если длительная операция запускается через интерфейс, пользователь должен иметь возможность остановить её, не закрывая приложение целиком.

3.9. Обработка ошибок

В асинхронных функциях CodeSlicer используется try/catch. Ошибка может возникнуть из-за недоступного local API, отсутствующей папки проекта, неверных параметров или внутреннего исключения backend.

Пример:

```
try {
   const response =
      await ImpactApi
         .graphProjection(
            payload
         );

   state.projection =
      response;

   renderProjection(
      response
   );
} catch (error) {
   showState(
      `Карта недоступна:
      ${error.message}`,
      'error-state'
   );
}
```

Пользователь получает фактическую ошибку операции, а интерфейс продолжает работать. Это более корректно, чем оставлять страницу в бесконечном состоянии загрузки.

4. Экранные формы с примерами выполнения программы

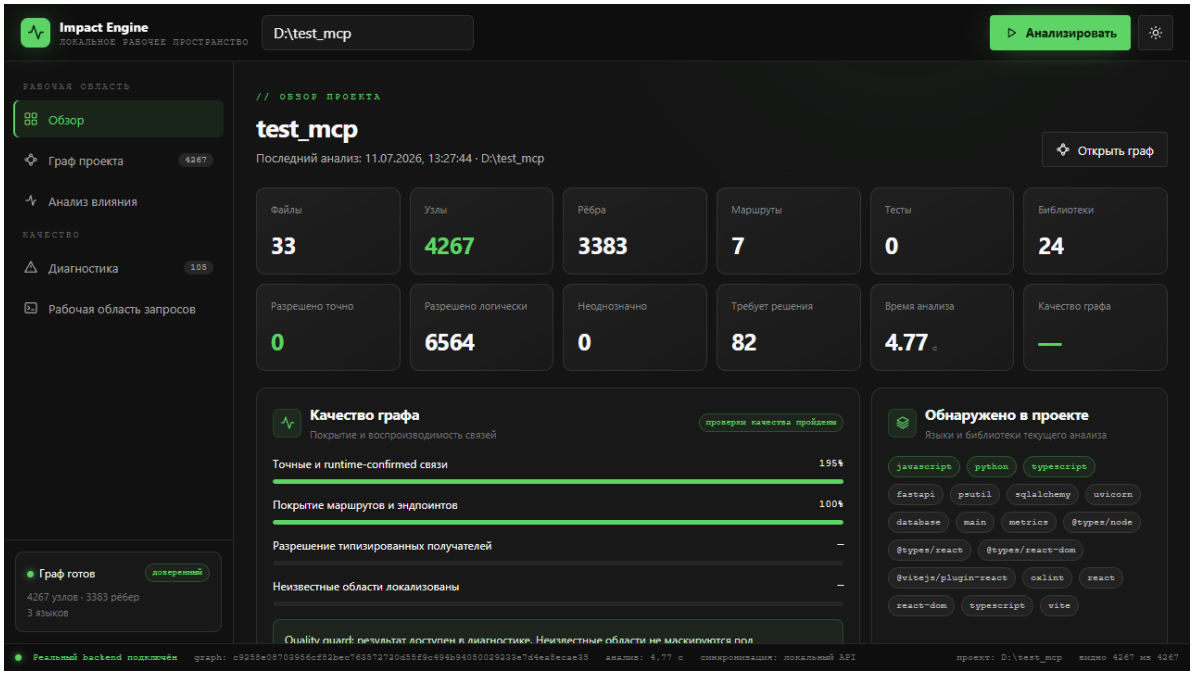


Рисунок 1 — карта проекта, загруженная асинхронно из local API.

Карта появляется после выполнения Promise и получения структуры узлов и связей. До этого интерфейс может отображать состояние загрузки.

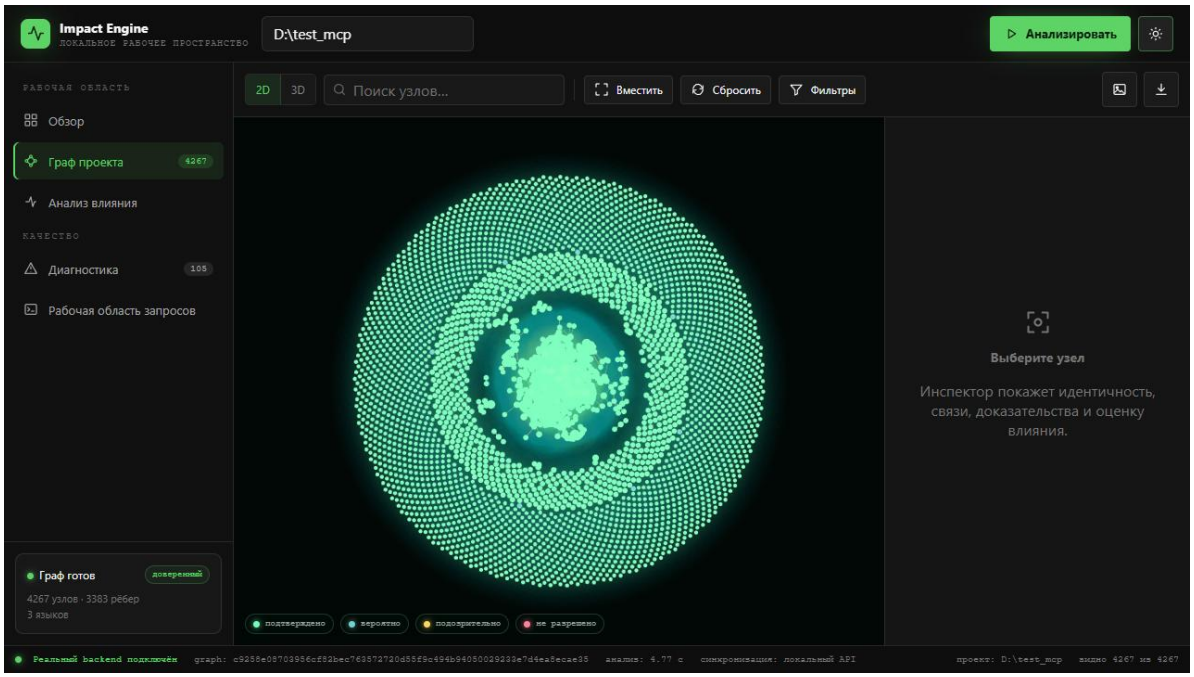


Рисунок 2 — интерактивное представление связей CodeSlicer.

После получения данных пользователь может масштабировать карту и переходить к отдельным сущностям. Эти действия выполняются без полной перезагрузки страницы.

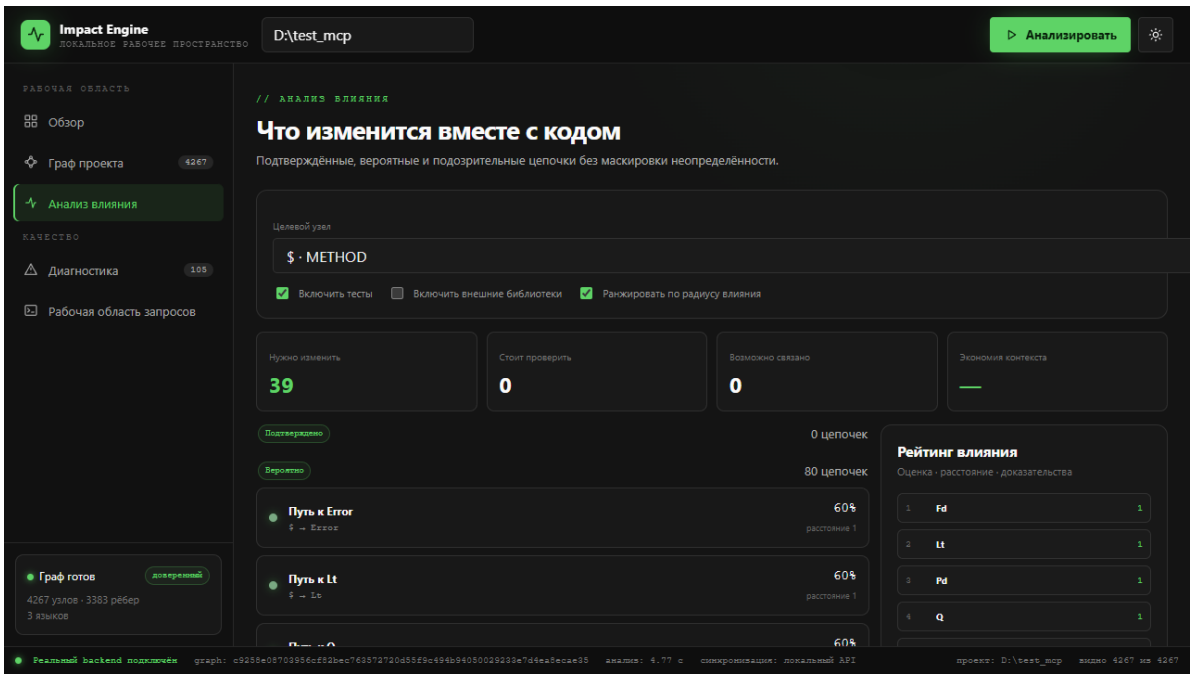


Рисунок 3 — результат review, полученный асинхронно.

Проверка изменений показывает, что асинхронная модель используется не только для простого GET-запроса, но и для более сложной серверной операции, возвращающей структурированный результат.

5. Вывод

В лабораторной работе рассмотрена асинхронная модель клиентской части CodeSlicer. Fetch API используется как основной транспорт, async/await делает последовательность операций читаемой, Promise.all ускоряет параллельные независимые запросы, а Promise.allSettled позволяет сохранить частичный результат при недоступности дополнительных источников.

На примере CodeSlicer видно, зачем асинхронность необходима в реальном приложении. Анализ проекта, построение графа и review не должны блокировать браузер. Пользователь получает состояние загрузки,

может продолжать взаимодействовать с интерфейсом и при необходимости отменить длительную операцию.

В результате требования лабораторной работы реализованы на полноценном веб-приложении: данные загружаются асинхронно, результат сохраняется в состоянии, ошибки обрабатываются, а DOM обновляется только после получения актуального ответа сервера.